

# CSE 220: Systems Fundamentals I

Stony Brook University

Homework 5

SPRING 2026

**Updated draft after feedback from the students on Piazza (3/17/26)**

**Due: Friday, March 27, 2026, at 11:59 PM EST**

## ● **IMPORTANT:**

- This homework will be marked out of 100 points.
- **Filename format: SPR26CSE220-<SBUusername>-<SBUID#>.zip**
- **You need to put your name and ID# in each file**
- **You need to use Criterion for testing**
- You need to submit a Makefile besides the C files, which the TAs will use to compile your code.
- You may add a README.txt file if you need to explain anything about your code.
- The code should follow proper software engineering conventions regarding indentation, variable naming, and commenting. 5% marks are assigned for this in each part.
- If any file is missing or incorrectly named, 10 % points will be deducted.
- **For each problem, you must add 10 additional test cases of your own in Criterion. These should be clearly documented in the test files.**

## Learning Outcomes

- Mastering C programming for pointers and pointer dereferencing

## Overview

In this assignment, you will be implementing a program to perform simple linear algebra computations: sparse matrix calculation, Hadamard product of matrices, matrix multiplication, and diagonal sum. The matrices and formulas containing these operations will be implemented using pointer dereferencing. You are not allowed to use arrays to implement the operation. You can initialize the matrices using arrays before calling the functions. The assumption is that all matrices have been initialized before calling the required functions. However, after passing matrices to the operations/functions, all calculations within the functions will be done using pointer dereferencing. **No marks will be given if a direct array approach is used.**

Although the matrix is declared as `int M[D[0]][D[1]]`, you must access elements using pointer arithmetic. For example, `M[i][j]` should be written as `*(*(M+i)+j)`.

Implement all operations in the `matrix.c` file. During grading, only your `matrix.c` file will be copied into the grading framework's directory for processing. Make sure all of your code is self-contained in that file.

## Mathematical Preliminaries

### Matrices

A matrix is a rectangular array of numbers arranged in rows and columns. The size of a matrix is determined by the number of rows and columns it contains. Matrices are widely used in mathematics, physics, engineering, computer science, and many other fields.

### Matrix Addition

Matrix addition is an operation that involves adding the corresponding elements of two matrices of the same size. Mathematically, if  $A$  and  $B$  are both  $m \times n$  matrices (with  $m$  rows and  $n$  columns), then their sum, denoted as  $C$ , is an  $m \times n$  matrix where the entry in the  $i$ -th row and  $j$ -th column of  $C$  is given by:

$$C_{ij} = A_{ij} + B_{ij}$$

In this formula,  $A_{ij}$  is the entry in the  $i$ -th row and  $j$ -th column of matrix  $A$ , and  $B_{ij}$  is the entry in the  $i$ -th row and  $j$ -th column of matrix  $B$ . Note that to add two matrices, they

must be of the same size, i.e., they must have the same number of rows and the same number of columns. You need to check this compatibility in your function as well.

## Matrix Multiplication

Matrix multiplication is a way of combining two matrices to produce a new matrix. The resulting matrix has the same number of rows as the first matrix and the same number of columns as the second matrix. Mathematically, if A is an  $m \times n$  matrix and B is an  $n \times p$  matrix, then the product of A and B, denoted as C, is an  $m \times p$  matrix where the entry in the  $i$ -th row and  $j$ -th column of C is given by:

$$C_{ij} = \sum_{k=1}^n (A_{ik} B_{kj})$$

In this formula,  $A_{ik}$  is the entry in the  $i$ -th row and  $k$ -th column of A, and  $B_{kj}$  is the entry in the  $k$ -th row and  $j$ -th column of B. In our scripts, matrix multiplication will be denoted  $C = A*B$ . ( In your function, you need to check if the number of columns of A is equal to the number of rows of B).

## Sparse Matrix

A matrix is a two-dimensional data object made of  $m$  rows and  $n$  columns, therefore having a total of  $m \times n$  values. If most of the elements of the matrix have a **0 value**, then it is called a sparse matrix. For this homework, we will consider a matrix to be sparse if the number of non-zero values is less than or equal to the largest dimension of the matrix.

### Why use a Sparse Matrix instead of a simple matrix?

- **Storage:** There are fewer non-zero elements than zeros, and thus, less memory can be used to store only those elements.
- **Computing time:** Computing time can be saved by logically designing a data structure that traverses only non-zero elements.

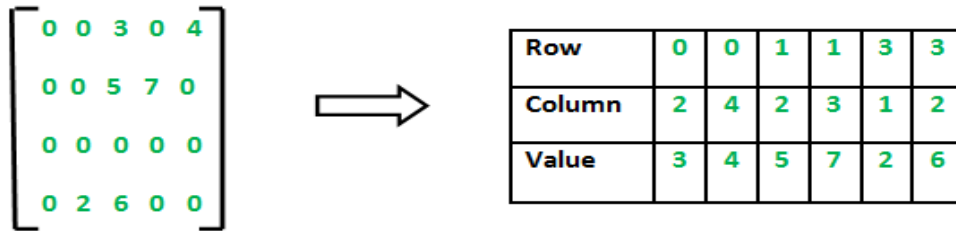


Figure -1 Sparse matrix representation. This figure is just to explain the concept of a sparse matrix. It does not satisfy our constraint. We define that the number of non-zero elements should be equal to or less than the largest dimension. Here, the largest dimension is 5. However, the nonzero elements are 6.

Sparse Matrix Representations can be done in many ways. The following are two common representations:

1. Array representation
2. Linked list representation

Since we have not yet covered the topic of advanced pointer usage with dynamic memory allocation, we will use the array representation for the sparse matrix representation in this homework.

A 2D array is used to represent a sparse matrix in which there are three rows named as:

- **Row:** Index of row, where a non-zero element is located
- **Column:** Index of column, where a non-zero element is located
- **Value:** Value of the non-zero element located at index - (row, column)
- See Figure 1.

## The Hadamard product

The **Hadamard product**, also known as the *Schur product* or element-wise product, is a binary operation that takes two matrices of the same dimensions and produces another matrix of identical size in which each entry is the product of the corresponding entries of the original matrices. Formally, for two matrices A and B (both of the same dimension  $m \times n$ ), the Hadamard product is denoted as

$$(A \odot B)_{ij} = a_{ij} \times b_{ij}, \quad \text{for all } 1 \leq i \leq m, 1 \leq j \leq n.$$

Unlike standard matrix multiplication, the Hadamard product does not involve row-column dot products and requires matrices to be strictly conformable in size. Its computational simplicity makes it widely used in applied mathematics and scientific computing. In particular, the Hadamard product plays a crucial role in **signal processing**, **image manipulation**, and **machine learning**, where element-wise

operations are common in neural network training (e.g., applying masks, gating mechanisms, and attention weight scaling). Additionally, it is employed in numerical linear algebra as a component in algorithms for covariance matrix estimation and in the modeling of component-wise interactions in statistics and econometrics.

## The Diagonal Sum

The **Diagonal Sum** operation is a matrix computation that focuses on aggregating values along specific structural lines of a matrix, most commonly its main diagonal, anti-diagonal, and cumulative row or column sums.

For a square matrix  $A$  of order  $n$ , several diagonal sums are defined that capture structural properties of the matrix.

Main diagonal sum:

$$\text{MainDiagonal}(A) = \sum_{i=1}^n a_{ii}$$

Anti-diagonal sum:

$$\text{AntiDiagonal}(A) = \sum_{i=1}^n a_{i, n-i+1}$$

Row sum for row  $i$ :

$$\text{RowSum}(i) = \sum_{j=1}^n a_{ij}$$

Column sum for column  $j$ :

$$\text{ColSum}(j) = \sum_{i=1}^m a_{ij}$$

These operations offer compact summaries of the distribution of values in a matrix and are widely used in **pattern recognition**, **image processing**, and **graph algorithms**, where diagonals capture structural symmetries and row/column sums highlight aggregate features. For example, in image analysis, diagonal and row/column sums are employed in feature extraction techniques, while in linear algebra, they can serve as diagnostic tools for verifying matrix properties (e.g., checking consistency or symmetry).

**Note:**

- Remember: In array notation,  $A[\text{row}][i]$  is equivalent to  $(*(A + \text{row}) + i)$ , and  $A[j]$  means  $*(A+j)$ ; You will be using such expressions in your homework.
- Remember: You will be using the Variable Length Arrays (VLA) concept defined by C99 in this assignment

You will implement the following functions:

1. `int SparseMatrix(int D[2], int M[D[0]][D[1]], int S[3][(D[1] > D[0] ? D[1] : D[0])]);`

M: 2D matrix which might have more zero elements than non-zero elements

S: 2D sparse matrix. However,  $3 \times m$ .  $m$  is equal to the largest dimension of M

D: The 1D matrix contains the dimensions of M. It's a two-element array. The first element gives the dimension along the y-axis, while the second element gives the dimension along the x-axis.

The function will return the number of non-zero elements if the number of non-zero elements is equal to or less than  $m$  ( the largest dimension of M). In this case, a Sparse matrix S will be created. Otherwise, the given M matrix is not sparse, and the function will return -1.

Note: In case the number of non-zero elements is less than the maximum dimension of M, the remaining elements of S will be filled with zeros. See Figure 2.

For this homework, the sparse matrix will have a  $3 \times m$  dimension. Where  $m$  is the maximum dimension of the given matrix (In case of our interface, it's S). For example, if we have 5 non-zero elements and the maximum dimension is 7, then  $S \rightarrow 3 \times 7$  as shown in Figure 2.

Note: Although you are just returning a number, we will test the  $S[][]$  matrix using a helper function to check your implementation. The function is provided to you as well.

|        |   |   |   |   |   |   |   |
|--------|---|---|---|---|---|---|---|
| rows   | 1 | 2 | 3 | 4 | 5 | 0 | 0 |
| Column | 1 | 2 | 3 | 4 | 5 | 0 | 0 |
| values | 3 | 3 | 3 | 3 | 3 | 0 | 0 |

Figure 2: The Yellow part is the real sparse matrix.

```
2. int HadamardProduct(const int D[6], const int M[D[0]][D[1]], const int N[D[2]][D[3]],
    int A[D[4]][D[5]]);
```

M, N, and A are 2D matrices. D is a 1D array with 1x6 elements, with the first two elements giving the dimensions of M (the first element gives the y-axis and the second element gives the x-axis), the next two elements giving the dimensions of N, and the last two elements giving the dimensions of A, respectively. Matrix A contains the outcome of the element-wise multiplication (Hadamard product) operation.

### Potential Cases

- Dimensions of M and N are compatible (M and N have the same dimensions) for algebraic element-wise multiplication of matrices:
  - A is compatible (A is the exact size of the result of element-wise multiplication). Straightforward operation. Matrix A will have the answer. **Return 1** (success).
  - If A is not compatible, it is oversized enough to contain the actual result. Matrix A will have the answer. **Return 2**.
  - If A is not compatible and does not have enough space to contain the whole result. In this case, it will contain part of the result. The one that overlaps with the space of the result/answer with the space of A. **Return -3**
- Dimensions of M and N are not compatible for algebraic element-wise multiplication (M and N have different dimensions). We cannot do the full element-wise multiplication. However, we move on with what we have and try to do the element-wise multiplication of elements that are possible.
  - Step 1: The space of M and N that overlaps should be multiplied element-wise. You can call it the intersection of the elements that are common to both matrices
    - $M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 3 & 6 \\ 7 & 8 & 9 \end{bmatrix} \rightarrow 3 \times 3$  matrix
    - $N = \begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix} \rightarrow 2 \times 2$  matrix
    - Neither is compatible. However, the first two rows and two columns overlap (call it the intersection of the two matrices). The elements of the common space will be multiplied element-wise.
    - $M \odot N \rightarrow \begin{bmatrix} (1 \times 2 = 2) & (2 \times 5 = 10) & (4 \times 1 = 4) & (3 \times 4 = 12) \end{bmatrix}$  will result in  $2 \times 2$  matrix
  - Matrix A will hold this result
  - If A has enough space or dimension to hold the result of element-wise multiplication from the previous step, A should hold it, and the function should **return -1**.
  - If A does not have enough space or dimension to hold the result of element-wise multiplication from the previous step, then A should hold the elements of the results from step 1 that overlap / intersect its dimension, and the function should **return -2**.

Note: Although you are just returning a number, we will test the A[][] matrix using a helper function to check your implementation. The function is provided to you as well.

Note: Matrix A should be initialized to zero. No garbage values.

3. `int Multiplication(const int D[6], const int M[D[0]][D[1]], const int N[D[2]][D[3]], int A[D[4]][D[5]]);`

M, N, and A are 2D matrices. D is a 1D array with 1x6 elements, with the first two elements giving the dimensions of M (the first element gives the y-axis and the second element gives the x-axis), the next two elements giving the dimensions of N, and the last two elements giving the dimensions of A, respectively. Matrix A contains the outcome of the multiplication operation.

### Potential Cases

- Dimensions of M and N are compatible (#cols of M = #rows of N) for algebraic multiplication of matrices:
  - If A is compatible (A is the exact dimension of the result of multiplication). Straightforward operation. Matrix A will have the answer. **Return 1** (success)
  - If A is not compatible, it is oversized enough to contain the actual result. Matrix A will have the answer. **Return 2.**
  - If A is not compatible, not enough space to contain the whole result. In this case, it will contain part of the result. The one that overlaps with the space of the result/answer. **Return -3.**
- Dimensions of M and N are not compatible for algebraic multiplication. However, we move on with what we have and try to do the multiplication (of elements) that is possible.
  - Step 1:
    - $M = \begin{bmatrix} 1, 2, 3 \\ 4, 3, 6 \\ 7, 8, 9 \end{bmatrix} \rightarrow 3 \times 3$  matrix
    - $N = \begin{bmatrix} 1, 2 \\ 3, 4 \end{bmatrix}$  (Hint: consider N to be  $\begin{bmatrix} 1, 2, 0 \\ 3, 4, 0 \end{bmatrix}$ )  $\rightarrow 2 \times 2$  matrix
    - $M \times N \rightarrow \begin{bmatrix} (1+6=7) & (2+8=10) \\ (4+9=13) & (8+12=20) \\ (7+24=31) & (14+32=46) \end{bmatrix}$  will result  $3 \times 2$  matrix
    - Matrix A will hold this result
  - If A has enough space or dimension to hold the result of multiplication from the previous step, A should hold it, and the function should **return -1.**
  - If A does not have enough space or dimension to hold the result of multiplication from the previous step, then A should hold the elements of the results from step 1 that overlap / intersect its dimension, and the function should **return -2.**

Note: Although you are just returning a number, we will test the A[][] matrix using a helper function to check your implementation. The function is provided to you as well.

Note: Matrix A should be initialized to zero. No garbage values

#### 4. `int DiagonalSum(const int D[4], const int A[D[0]][D[1]], int DS[D[2]][D[3]])`

A and DS are 2D matrices. D is a 1D array with 1×4 elements, with the first two elements giving the dimension of A (the first element gives the y-axis and the second element gives the x-axis), and the next two elements giving the dimension of DS. Matrix DS contains the outcome of the diagonal sum operation. The diagonal sum includes the main diagonal sum, the anti-diagonal sum for square matrices, as well as the sums of each row and each column. DS is structured so that DS[0][0] stores the main diagonal sum, DS[0][1] stores the anti-diagonal sum, DS[1][j] stores the column sums, and DS[i+2][0] stores the row sums.

The function will return 1 if the dimensions of the matrices are compatible (DS has sufficient space to store all diagonal, row, and column sums). If dimensions are not compatible, then:

- If DS is oversized but still has enough capacity to store the required results, DS should hold all results, and the function should return 2.
- If DS does not have enough space or dimension to hold the full result, then DS should store the overlapping part of the result, and the function should return -1

Note: Matrix DS should be initialized to zero. No garbage values

## Test cases:

`int SparseMatrix(int D[2], int M[D[0]][D[1]], int S[3][(D[1] > D[0] ? D[1] : D[0])]);`

Case 1:

$$M = \begin{bmatrix} 0 & 4 \\ 0 & 5 \\ 0 & 0 \\ 2 & 0 \end{bmatrix}$$

4×2

So Sparse Matrix Dimension,  $S = 3 \times m$ , hence  $m = 4$ , and the number of non-zero elements is 3, which is less than  $m$ . So this matrix satisfies the sparse matrix condition/definition, and it's compatible with creating sparse matrices. **It returns 3.**

Hence, the sparse matrix dimension is  $S = 3 \times 4$ .

|        |   |   |   |   |
|--------|---|---|---|---|
| Row    | 0 | 1 | 3 | 0 |
| Column | 1 | 1 | 0 | 0 |
| Value  | 4 | 5 | 2 | 0 |

**Note:** In column 4, all elements are 0 as no non-zero value is available to put there.

Case 2:

$$M = \begin{bmatrix} 2 & 3 & 0 & 5 \\ 3 & 0 & 3 & 1 \\ 4 & 0 & 3 & 0 \\ 0 & 0 & 6 & 0 \end{bmatrix}_{4 \times 4}$$

Sparse Matrix Dimension  $3 \times m$ , hence  $m=4$ , and the number of non-zero elements is 9, which is higher than  $m$ . Therefore, this matrix does not satisfy the sparse matrix condition. **So the function returns -1.**

**int HadamardProduct(const int D[6], const int M[D[0]][D[1]], const int N[D[2]][D[3]], int A[D[4]][D[5]]);**

Case 1:

| M                                                                      | N                                                                      | A                                                                          | D                    | Compatible/Return                          |
|------------------------------------------------------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------|----------------------|--------------------------------------------|
| $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$<br>$2 \times 3$ | $\begin{bmatrix} 2 & 3 & 4 \\ 5 & 6 & 7 \end{bmatrix}$<br>$2 \times 3$ | $\begin{bmatrix} 2 & 6 & 12 \\ 20 & 30 & 42 \end{bmatrix}$<br>$2 \times 3$ | $[2, 3, 2, 3, 2, 3]$ | Yes/Return 1<br>M, N, and A are compatible |

**Case 2:**

| M                                                           | N                                                           | A                                                                                  | D                    | Compatible/Return                                                                                                         |
|-------------------------------------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------|----------------------|---------------------------------------------------------------------------------------------------------------------------|
| $\begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix}$ $2 \times 2$ | $\begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix}$ $2 \times 2$ | $\begin{bmatrix} 3 & 8 & 0 \\ 35 & 48 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ $3 \times 3$ | $[2, 2, 2, 2, 3, 3]$ | Return 2<br><br>M and N are compatible, but not A. However, A can hold the product<br>Compatible, oversized result matrix |

**Case 3:**

| M                                                                   | N                                                                   | A                                                                                                                             | D                    | Compatible/Return                                                 |
|---------------------------------------------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|----------------------|-------------------------------------------------------------------|
| $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ $2 \times 3$ | $\begin{bmatrix} 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ $2 \times 3$ | $\begin{array}{ccc ccc} \hline 3 & 16 & 18 & \hline 28 & 40 & 54 & \hline 0 & 0 & 0 & \hline \hline \end{array}$ $3 \times 3$ | $[2, 3, 2, 3, 3, 3]$ | Return 2<br><br>Compatible, but intersection works (A big enough) |

**Case 4:**

| M                                                                                | N                                                           | A                                                                                  | D                    | Compatible/Return                                                       |
|----------------------------------------------------------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------|----------------------|-------------------------------------------------------------------------|
| $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ $3 \times 3$ | $\begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix}$ $2 \times 2$ | $\begin{bmatrix} 2 & 10 & 0 \\ 4 & 20 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ $3 \times 3$ | $[3, 3, 2, 2, 3, 3]$ | No/Return -1<br><br>Incompatible, but intersection works (A big enough) |

**Case 5:**

| M                                                                                | N                                                           | A                                                             | D                    | Compatible/Return                                                     |
|----------------------------------------------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------|----------------------|-----------------------------------------------------------------------|
| $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ $3 \times 3$ | $\begin{bmatrix} 2 & 5 \\ 1 & 4 \end{bmatrix}$ $2 \times 2$ | $\begin{bmatrix} 2 & 10 \\ 4 & 20 \end{bmatrix}$ $2 \times 2$ | $[3, 3, 2, 2, 2, 2]$ | Return - 2<br><br>Incompatible, intersection<br>only partially stored |

**int Multiplication(const int D[6], const int M[D[0]][D[1]], const int N[D[2]][D[3]], int A[D[4]][D[5]]);**

**Case 1:**

| M                                                                    | N                                                                   | D                         | A                                                                                       | Compatible/Return |
|----------------------------------------------------------------------|---------------------------------------------------------------------|---------------------------|-----------------------------------------------------------------------------------------|-------------------|
| $\begin{bmatrix} 2 & 3 \\ 5 & 4 \\ 2 & 3 \end{bmatrix}$ $3 \times 2$ | $\begin{bmatrix} 3 & 2 & 1 \\ 1 & 3 & 4 \end{bmatrix}$ $2 \times 3$ | $[3 \ 2 \ 2 \ 3 \ 3 \ 3]$ | $\begin{bmatrix} 9 & 13 & 14 \\ 19 & 22 & 21 \\ 9 & 13 & 14 \end{bmatrix}$ $3 \times 3$ | Yes/Return 1      |

**Case 2:**

| M                                                                    | N                                                                    | D                         | A                                                               | Compatible/Return                                                  |
|----------------------------------------------------------------------|----------------------------------------------------------------------|---------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------|
| $\begin{bmatrix} 2 & 3 \\ 5 & 4 \\ 2 & 3 \end{bmatrix}$ $3 \times 2$ | $\begin{bmatrix} 3 & 3 \\ 7 & 2 \\ 1 & 5 \end{bmatrix}$ $3 \times 2$ | $[3 \ 2 \ 3 \ 2 \ 2 \ 2]$ | $\begin{bmatrix} 27 & 12 \\ 43 & 23 \end{bmatrix}$ $2 \times 2$ | <b>No/ Return -1</b><br><br>A can hold part of<br>the final result |

**Case 3:**

| M                                                                    | N                                                                    | D                         | A                                                                                                   | Compatible/Return                                      |
|----------------------------------------------------------------------|----------------------------------------------------------------------|---------------------------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| $\begin{bmatrix} 2 & 3 \\ 5 & 4 \\ 2 & 3 \end{bmatrix}$ $3 \times 2$ | $\begin{bmatrix} 3 & 3 \\ 7 & 2 \\ 1 & 5 \end{bmatrix}$ $3 \times 2$ | $[3 \ 2 \ 3 \ 2 \ 4 \ 3]$ | $\begin{bmatrix} 27 & 12 & 0 \\ 43 & 23 & 0 \\ 27 & 12 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ $4 \times 3$ | No/ Return -1<br><br>A can hold the complete<br>result |

**Case 4:**

| M                                                                    | N                                                                   | D             | A                                                                                                                    | Compatible/Return                                                               |
|----------------------------------------------------------------------|---------------------------------------------------------------------|---------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| $\begin{bmatrix} 2 & 3 \\ 5 & 4 \\ 2 & 3 \end{bmatrix}$ $3 \times 2$ | $\begin{bmatrix} 3 & 2 & 1 \\ 1 & 3 & 4 \end{bmatrix}$ $2 \times 3$ | [3 2 2 3 4 4] | $\begin{bmatrix} 9 & 13 & 14 & 0 \\ 19 & 22 & 19 & 0 \\ 9 & 13 & 14 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$ $4 \times 4$ | Return 2<br><br>M and N are compatible<br>A will have the right/complete result |

**Case 5:**

| M                                                                    | N                                                                   | D             | A                                                                                | Compatible/Return                                                                      |
|----------------------------------------------------------------------|---------------------------------------------------------------------|---------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| $\begin{bmatrix} 2 & 3 \\ 5 & 4 \\ 2 & 3 \end{bmatrix}$ $3 \times 2$ | $\begin{bmatrix} 3 & 2 & 1 \\ 1 & 3 & 4 \end{bmatrix}$ $2 \times 3$ | [3 2 2 3 2 4] | $\begin{bmatrix} 9 & 13 & 14 & 0 \\ 19 & 22 & 19 & 0 \end{bmatrix}$ $2 \times 4$ | Return -3<br><br>M and N are compatible<br>A does not have the right result dimensions |

**DiagonalSum(const int D[4], const int A[D[0]][D[1]], int DS[D[2]][D[3]])**

**Case 1:**

| A                                                                                | DS                                                                                                                                    | D            | Compatible/Return                             |
|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|--------------|-----------------------------------------------|
| $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ $3 \times 3$ | $\begin{bmatrix} 15 & 15 & 0 & 0 \\ 12 & 15 & 18 & 0 \\ 6 & 0 & 0 & 0 \\ 15 & 0 & 0 & 0 \\ 24 & 0 & 0 & 0 \end{bmatrix}$ $5 \times 4$ | [3, 3, 5, 4] | DS Oversized<br>Return 2<br><br>Square matrix |

**Case 2:**

| A                                                         | DS                                                                                                                                 | D              | Compatible/Return                               |
|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|----------------|-------------------------------------------------|
| $\begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}$ <p>2x2</p> | $\begin{bmatrix} 10 & 10 & 0 & 0 \\ 8 & 12 & 0 & 0 \\ 6 & 0 & 0 & 0 \\ 14 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$ <p>5 x 5</p> | $[2, 2, 5, 4]$ | <p>2x2 matrix, DS oversized</p> <p>return 2</p> |

**Case 3:**

| A                                                                                                           | DS                                                              | D           | Compatible/Return                                                                            |
|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|-------------|----------------------------------------------------------------------------------------------|
| $\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \end{bmatrix}$ <p>5x5</p> | $\begin{bmatrix} 16 & 16 \\ 19 & 14 \end{bmatrix}$ <p>2 x 2</p> | $[5,5,2,2]$ | <p>No/ Return -1</p> <p>5x5 matrix, DS too small</p> <p>Does not have the correct result</p> |

**Case 4:**

| A                                                                 | DS                                                                                               | D           | Compatible/Return                                  |
|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-------------|----------------------------------------------------|
| $\begin{bmatrix} 3 & 5 & 7 \\ 2 & 4 & 6 \end{bmatrix}$ <p>3x2</p> | $\begin{bmatrix} 7 & 9 & 0 \\ 5 & 9 & 13 \\ 15 & 0 & 0 \\ 12 & 0 & 0 \end{bmatrix}$ <p>4 x 3</p> | $[3,2,4,3]$ | <p>Yes/ Return 1</p> <p>Rectangular 2x3 matrix</p> |

**Case 5:**

| A                                                                                           | DS                                                             | D              | Compatible/Return                                                |
|---------------------------------------------------------------------------------------------|----------------------------------------------------------------|----------------|------------------------------------------------------------------|
| $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 1 & 2 & 3 \end{bmatrix}$ <p>4x3</p> | $\begin{bmatrix} 15 & 9 \\ 17 & 17 \end{bmatrix}$ <p>2 x 2</p> | $[4, 3, 2, 2]$ | <p>No/ Return -1</p> <p>Rectangular 4x3 matrix, DS too small</p> |



## HINTS- Optional. You do not need to think this way

### **Multiplication:**

Initialize matrix A with zeros using pointer dereferencing.

Determine the effective multiplication limits.

Standard matrix multiplication requires:

$mCols == nRows$

If the dimensions match, perform full multiplication.

If they do not match, compute the multiplication using the overlapping inner dimension:

$kLimit = \min(mCols, nRows)$

Only compute results that fit inside matrix A.

### **DiagonalSum**

Initialize DS with zeros using pointer dereferencing.

Compute the main diagonal sum:

elements where  $row\ index == column\ index$ .

If the matrix is square, also compute the anti-diagonal sum:

elements where  $row + column == size - 1$ .

Compute the sum of each column.

Compute the sum of each row.

Store all computed values inside matrix DS.

## Running, Testing, and Submitting Your Code

Running test cases using Criterion is not mandatory for this homework. You can have your way of testing the code. However, as we move to the next assignment, we will adopt this framework for testing the homework. The testing framework is provided in the starter code. Try it and familiarize yourself.

To run the provided unit tests (written using [Criterion](#)):

- make to build your code
- make test to test your code

The provided test cases are nowhere near comprehensive. It is your responsibility to write additional test cases to verify the correctness of your code. If you create new test cases using Criterion (instead of printf), then both make tests.

## Grading Notes

During grading, only your matrix.c file will be copied into the grading framework's directory for processing. Make sure all of your code is self-contained in that file.

**Software Engineering = 5%**

**Your tests = 15 %**

**TA tests = 85%**

## Academic Honesty Policy

Academic honesty is taken very seriously in this course. By submitting your work for grading, you indicate your understanding of, and agreement with, the following Academic Honesty Statement:

1. I understand that representing another person's work as my own is academically dishonest.
2. I understand that copying, even with modifications, a solution from another source (such as the web or another person) as a part of my answer constitutes plagiarism.
3. I understand that sharing parts of my homework solutions (text write-up, schematics, code, electronic or hard-copy) is academic dishonesty and helps others plagiarize my work.
4. I understand that protecting my work from possible plagiarism is my responsibility. I understand the importance of saving my work so that it is visible only to me.
5. I understand that passing information that is relevant to a homework/exam to others in the course (either lecture or even in the future!) for their private use constitutes academic dishonesty. I will only discuss material that I am willing to openly post on the discussion board.
6. I understand that academic dishonesty is treated very seriously in this course. I understand that the instructor will report any incident of academic dishonesty to the University's Academic Judiciary.
7. I understand that the penalty for academic dishonesty might not be immediately administered. For instance, cheating on a homework assignment may be discovered and penalized after the grades for that homework have been recorded.
8. I understand that buying or paying another entity for any code, partial or in its entirety, and submitting it as my own work is considered academic dishonesty.
9. I understand that there are no extenuating circumstances for academic dishonesty.